

GitHub & Jira Ingestion — Complete Architecture

Platform: Enterprise AI Knowledge Platform

Scope: GitHub OAuth App + Jira OAuth 2.0 (3LO) — Authorization, Connection, Initial Sync, Ongoing Ingestion

Stack: Next.js (frontend) · FastAPI (backend) · PostgreSQL (Supabase) · Redis Streams · Cognee (KuzuDB + LanceDB)

Overview

The ingestion pipeline for GitHub and Jira is divided into four stages that happen sequentially per user/org connection:

1. **Authorization** — user logs into your platform, clicks "Connect GitHub / Jira", completes OAuth, your backend stores tokens securely
2. **Connection Setup** — once tokens are stored, create a connection record, register webhooks on the provider, and mark the connection as active
3. **Initial Snapshot** — a one-time full historical pull of all repos, issues, PRs, sprints, etc.
4. **Ongoing Ingestion** — real-time updates via webhooks + periodic polling as a safety net

1. Authorization Flow

1a. GitHub — OAuth Web Application Flow

GitHub recommends using a **GitHub App** over a plain OAuth App for production SaaS integrations because GitHub Apps:

- Use **installation-level tokens** (scoped per org or repo), not user-level tokens
- Support **fine-grained permissions** and short-lived tokens
- Can receive webhooks at the app level (no need to register per repo)

The flow from user's perspective:

```
sequenceDiagram
    actor U as User (Browser)
    participant FE as Next.js Frontend
    participant BE as FastAPI Backend
    participant GH as GitHub

    U->>FE: Clicks "Connect GitHub"
    FE->>BE: GET /api/integrations/github/authorize
    BE->>BE: Generate state = HMAC(user_id + timestamp)\nStore state in Redis (TTL: 5m)
    BE-->>FE: Redirect URL
```

```

FE-->GH: GET https://github.com/login/oauth/authorize\n?client_id=YOUR_CLIENT_ID'
GH-->U: GitHub consent screen\n(shows requested scopes)
U-->GH: Click "Authorize"
GH-->FE: GET /callback/github?code=AUTH_CODE&state=STATE_TOKEN
FE-->BE: POST /api/integrations/github/callback\n{code, state}
BE-->BE: Verify state matches Redis record\n(CSRF protection)
BE-->GH: POST https://github.com/login/oauth/access_token\n{client_id, client_secret, code, state}
GH-->BE: {access_token, scope, token_type}
BE-->GH: GET https://api.github.com/user\nAuthorization: Bearer ACCESS_TOKEN
GH-->BE: {id, login, email, name}
BE-->BE: Encrypt and store token\nCreate connection record in Postgres
BE-->FE: {status: "connected", connection_id}
FE-->U: "GitHub connected ✓"

```

GitHub Scopes Required

Scope	Why needed
repo	Read access to repos, issues, PRs, commits
read:org	Read organization membership and teams
read:user	Read user profile (for identity resolution across tools)
read:project	Read GitHub Projects (optional, for project board data)
admin:repo_hook	Register webhooks on repos programmatically

Token Exchange — Exact API Calls

```

# Step 1: Redirect user to GitHub
import secrets, hashlib, base64

def generate_pkce_pair():
    verifier = secrets.token_urlsafe(43)
    challenge = base64.urlsafe_b64encode(
        hashlib.sha256(verifier.encode()).digest()
    ).rstrip(b'=').decode()
    return verifier, challenge

# Step 2: Exchange code for token (FastAPI callback handler)
async def github_callback(code: str, state: str, db: Session, redis: Redis):
    # Validate state
    stored_state = await redis.get(f"oauth_state:{state}")
    if not stored_state:
        raise HTTPException(403, "Invalid or expired state")

    # Exchange code for access token
    response = await httpx.post(
        "https://github.com/login/oauth/access_token",
        json={
            "client_id": settings.GITHUB_CLIENT_ID,
            "client_secret": settings.GITHUB_CLIENT_SECRET,
            "code": code,
            "redirect_uri": settings.GITHUB_CALLBACK_URL,

```

```

        "code_verifier": stored_state["pkce_verifier"]
    },
    headers={"Accept": "application/json"}
)
token_data = response.json()
access_token = token_data["access_token"]

# Fetch user identity
user_resp = await httpx.get(
    "https://api.github.com/user",
    headers={"Authorization": f"Bearer {access_token}"})
gh_user = user_resp.json()

# Store encrypted token in Postgres
connection = IntegrationConnection(
    user_id=current_user.id,
    provider="github",
    provider_account_id=str(gh_user["id"]),
    provider_username=gh_user["login"],
    access_token=encrypt(access_token), # AES-256 encryption
    scopes=token_data["scope"],
    status="pending_setup",
)
db.add(connection)
db.commit()

# Trigger async setup (webhook registration + initial sync)
await queue.enqueue("setup_github_connection", connection.id)
return {"status": "connected", "connection_id": connection.id}

```

1b. Jira — OAuth 2.0 (3LO) Flow

Jira uses Atlassian's hosted auth server at auth.atlassian.com. The flow is nearly identical to GitHub's but requires an extra step after the token exchange: fetching the **cloudId** of the Jira site before any API calls.

```

sequenceDiagram
    actor U as User (Browser)
    participant FE as Next.js Frontend
    participant BE as FastAPI Backend
    participant ATL as Atlassian Auth Server
    participant JR as Jira REST API

    U->>FE: Clicks "Connect Jira"
    FE->>BE: GET /api/integrations/jira/authorize
    BE->>BE: Generate state token\nStore in Redis (TTL: 10 min)
    BE-->>FE: Redirect URL
    FE->>ATL: GET https://auth.atlassian.com/authorize\n?audience=api.atlassian.com\n
    ATL-->>U: Atlassian consent screen\n(user selects which site to grant)
    U->>ATL: Click "Accept"
    ATL->>FE: GET /callback/jira?code=AUTH_CODE&state=STATE
    FE->>BE: POST /api/integrations/jira/callback {code, state}

```

```

BE-->BE: Verify state from Redis
BE-->ATL: POST https://auth.atlassian.com/oauth/token\n{grant_type: authorization
ATL-->BE: {access_token, refresh_token, expires_in, scope}
BE-->JR: GET https://api.atlassian.com/oauth/token/accessible-resources\nAuthoriz
JR-->BE: [{id: "cloudId", name: "site-name", url: "..."}]
BE-->JR: GET https://api.atlassian.com/me\n(get Atlassian account_id for identity
JR-->BE: {account_id, email, name}
BE-->BE: Encrypt and store tokens + cloudId\nCreate connection record
BE-->FE: {status: "connected", sites: [...]}
FE-->U: "Jira connected ✓ – site: your-org.atlassian.net"

```

Jira Scopes Required

Scope	Why needed
read:jira-work	Read issues, sprints, epics, projects, comments
read:jira-user	Read user profiles (identity resolution)
write:jira-work	Optional — only if you plan to create issues from the platform
offline_access	Required for refresh tokens — without this, tokens expire in 1h with no way to renew

Critical: Always include `offline_access` in your Jira scope. Jira access tokens expire in ~60 minutes. Without a refresh token your connection dies after the first hour.

Token Storage Schema (Postgres)

```

CREATE TABLE integration_connections (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID NOT NULL REFERENCES users(id),
  tenant_id         UUID NOT NULL REFERENCES tenants(id),
  provider          TEXT NOT NULL,           -- 'github' | 'jira'
  status            TEXT NOT NULL DEFAULT 'pending_setup',
                                           -- pending_setup | active | error | re
  provider_account_id TEXT,                 -- GitHub user ID or Jira account_id
  provider_username  TEXT,                 -- GitHub login or Jira email

  -- Jira-specific
  cloud_id          TEXT,                   -- Jira cloudId (required for all API
  site_url          TEXT,                   -- https://org.atlassian.net
  site_name         TEXT,

  -- Tokens (AES-256 encrypted at app layer, stored as bytea)
  access_token_enc   BYTEA NOT NULL,
  refresh_token_enc  BYTEA,                 -- NULL for GitHub, set for Jira
  token_expires_at   TIMESTAMPTZ,         -- NULL for GitHub (long-lived), set
  scopes            TEXT,

  -- Sync state
  initial_sync_status TEXT DEFAULT 'pending', -- pending | in_progress | completed
  initial_sync_started_at TIMESTAMPTZ,
  initial_sync_completed_at TIMESTAMPTZ,
  last_event_received_at TIMESTAMPTZ,

```

```

last_poll_cursor          TEXT,                -- updated_at timestamp or since_id for
-- Webhook
webhook_id                TEXT,                -- provider-side webhook ID (for dereg
webhook_secret            TEXT,                -- HMAC secret for signature verificat

created_at                TIMESTAMPTZ DEFAULT NOW(),
updated_at                TIMESTAMPTZ DEFAULT NOW()
);

```

2. Token Refresh Strategy

Jira tokens expire in ~60 minutes. GitHub tokens are long-lived but can be revoked. Both need a robust refresh/retry cycle.

```

flowchart TD
    API[API Call Needed] --> CHECK{Token expired?\nor expiring in < 5 min?}
    CHECK -- No --> CALL[Make API Call]
    CALL --> RESP{Response}
    RESP -- 200 OK --> DONE[Process response]
    RESP -- 401 Unauthorized --> REFRESH
    CHECK -- Yes --> REFRESH[Refresh Token Flow]

    subgraph REFRESH_FLOW["Refresh Token Flow (Jira)"]
        REFRESH --> POST["POST https://auth.atlassian.com/oauth/token\n{grant_type: refresh_token}"]
        POST --> NEW{Success?}
        NEW -- Yes --> STORE["Store new access_token\n+ new refresh_token\nUpdate token"]
        STORE --> RETRY[Retry original API call]
        NEW -- No 403 invalid_grant --> REVOKE["Mark connection status = revoked\nNot"]
    end
    end

```

```

# Token refresh middleware – wraps all provider API calls
class JiraClient:
    async def get(self, path: str, connection: IntegrationConnection):
        token = decrypt(connection.access_token_enc)

        if connection.token_expires_at < datetime.utcnow() + timedelta(minutes=5):
            token = await self._refresh_token(connection)

        response = await httpx.get(
            f"https://api.atlassian.com/ex/jira/{connection.cloud_id}/{path}",
            headers={"Authorization": f"Bearer {token}", "Accept": "application/json"}
        )

        if response.status_code == 401:
            # Token silently expired or revoked
            token = await self._refresh_token(connection)
            response = await httpx.get(...) # retry once

        return response

    async def _refresh_token(self, connection: IntegrationConnection):

```

```

resp = await httpx.post("https://auth.atlassian.com/oauth/token", json={
    "grant_type": "refresh_token",
    "client_id": settings.JIRA_CLIENT_ID,
    "client_secret": settings.JIRA_CLIENT_SECRET,
    "refresh_token": decrypt(connection.refresh_token_enc)
})
if resp.status_code != 200:
    await self._mark_revoked(connection)
    raise TokenRevokedException()

data = resp.json()
# IMPORTANT: Jira uses rotating refresh tokens – always replace both tokens
connection.access_token_enc = encrypt(data["access_token"])
connection.refresh_token_enc = encrypt(data["refresh_token"])
connection.token_expires_at = datetime.utcnow() + timedelta(seconds=data["exp"])
db.commit()
return data["access_token"]

```

3. Connection Setup (Post-Authorization)

After OAuth completes, a background job runs `setup_connection`:

```

flowchart TD
    TRIGGER[Queue: setup_github_connection / setup_jira_connection]

    subgraph SETUP["Connection Setup Worker"]
        direction TB
        A[Load connection record]
        B[Register webhook on provider]
        C[Store webhook_id + secret in Postgres]
        D[Update status = active]
        E[Enqueue: initial_sync job]
    end

    TRIGGER --> A --> B --> C --> D --> E

```

3a. GitHub — Register Webhooks

```

async def setup_github_connection(connection_id: str):
    conn = db.get(IntegrationConnection, connection_id)
    token = decrypt(conn.access_token_enc)

    # Fetch all orgs/repos the user has access to
    orgs = await github_client.get("https://api.github.com/user/orgs", token)

    # Register org-level webhook (covers all repos in the org)
    for org in orgs:
        webhook_secret = secrets.token_hex(32)
        resp = await httpx.post(
            f"https://api.github.com/orgs/{org['login']}/hooks",
            headers={"Authorization": f"Bearer {token}"},
            json={

```

```

        "name": "web",
        "active": True,
        "events": [
            "push",
            "pull_request",
            "issues",
            "issue_comment",
            "create",
            "delete",
            "deployment",
            "deployment_status",
            "repository",
            "member"
        ],
        "config": {
            "url": f"https://api.yourdomain.com/webhooks/github/{conn.id}",
            "content_type": "json",
            "secret": webhook_secret
        }
    }
)
# Store webhook registration
db.add(WebhookRegistration(
    connection_id=conn.id,
    provider="github",
    provider_webhook_id=str(resp.json()["id"]),
    secret_enc=encrypt(webhook_secret),
    scope=f"org:{org['login']}"
))

conn.status = "active"
db.commit()
await queue.enqueue("initial_sync_github", conn.id)

```

3b. Jira — Register Webhooks

```

async def setup_jira_connection(connection_id: str):
    conn = db.get(IntegrationConnection, connection_id)
    token = decrypt(conn.access_token_enc)
    cloud_id = conn.cloud_id

    webhook_secret = secrets.token_hex(32)

    # Register dynamic webhook (Jira Cloud supports up to 30 dynamic webhooks per app)
    resp = await httpx.post(
        f"https://api.atlassian.com/ex/jira/{cloud_id}/rest/api/3/webhook",
        headers={"Authorization": f"Bearer {token}", "Accept": "application/json",
                "Content-Type": "application/json"},
        json={
            "url": f"https://api.yourdomain.com/webhooks/jira/{conn.id}",
            "webhooks": [
                {
                    "events": [
                        "jira:issue_created",
                        "jira:issue_updated",

```

```

        "jira:issue_deleted",
        "comment_created",
        "comment_updated",
        "sprint_started",
        "sprint_closed",
        "sprint_updated"
    ],
    "jqlFilter": "project is not EMPTY"    # all projects
}
]
}
)

# NOTE: Jira dynamic webhooks expire after 30 days – must be refreshed
webhook_data = resp.json()
for wh in webhook_data.get("webhookRegistrationResult", []):
    db.add(WebhookRegistration(
        connection_id=conn.id,
        provider="jira",
        provider_webhook_id=str(wh["createdWebhookId"]),
        secret_enc=encrypt(webhook_secret),
        expires_at=datetime.utcnow() + timedelta(days=28)    # refresh before 30-d
    ))

conn.status = "active"
db.commit()
await queue.enqueue("initial_sync_jira", conn.id)

```

Jira webhook expiry warning: Jira dynamic webhooks expire after 30 days. You must have a scheduled job that calls `PUT /rest/api/3/webhook/refresh` every 25–28 days per active connection, otherwise webhooks stop firing silently.

4. Initial Snapshot (Full Historical Pull)

The initial sync is a paginated, cursor-based full pull of all historical data. It runs once per new connection, writes to S3 (raw archive), normalises, and ingests into Cognee.

```

flowchart TD
    START[Initial Sync Job Triggered]

    subgraph GITHUB_SYNC ["GitHub Initial Sync"]
        direction TB
        G1[Fetch all repos in org\nGET /orgs/:org/repos?per_page=100&page=N]
        G2[For each repo: fetch issues\nGET /repos/:owner/:repo/issues?state=all&per_page=100]
        G3[For each repo: fetch PRs\nGET /repos/:owner/:repo/pulls?state=all&per_page=100]
        G4[For each repo: fetch commits\nGET /repos/:owner/:repo/commits?per_page=100]
        G5[Fetch org members\nGET /orgs/:org/members]
        G1 --> G2 --> G3 --> G4 --> G5
    end

    subgraph JIRA_SYNC ["Jira Initial Sync"]
        direction TB
        J1[Fetch all projects\nGET /rest/api/3/project/search?maxResults=50&startAt=0]
    end

```



```

J2[For each project: fetch issues via JQL\nGET /rest/api/3/search?jql=project=
J3[Fetch all sprints\nGET /rest/agile/1.0/board/:boardId/sprint]
J4[Fetch all users\nGET /rest/api/3/users/search?maxResults=50]
J1 --> J2 --> J3 --> J4
end

subgraph INGEST["Ingest Pipeline (per batch)"]
  direction TB
  P1[Archive raw JSON to S3\ns3://bucket/raw/{provider}/{conn_id}/{resource}/{id}.json]
  P2[Normalise to internal schema]
  P3[Entity resolution\nGitHub user == Jira user via email]
  P4[Extract cross-source relationships]
  P5[add_data_points to Cognee\nscoped to dataset: tenant_id]
  P6[Update sync cursor in Postgres]
  P1 --> P2 --> P3 --> P4 --> P5 --> P6
end

START --> GITHUB_SYNC
START --> JIRA_SYNC
GITHUB_SYNC --> INGEST
JIRA_SYNC --> INGEST

```

Pagination Pattern

Both GitHub and Jira use cursor/offset pagination. Always page until empty:

```

# GitHub – Link header pagination
async def paginate_github(url: str, token: str):
    results = []
    while url:
        resp = await httpx.get(url, headers={"Authorization": f"Bearer {token}"})
        resp.raise_for_status()
        results.extend(resp.json())

        # GitHub returns next page URL in Link header
        link_header = resp.headers.get("Link", "")
        next_url = None
        for part in link_header.split(","):
            if 'rel="next"' in part:
                next_url = part.split(";")[0].strip().strip("<>")
        url = next_url

    return results

# Jira – startAt/total offset pagination
async def paginate_jira(path: str, token: str, cloud_id: str, params: dict = {}):
    results = []
    start_at = 0
    max_results = 100

    while True:
        resp = await httpx.get(
            f"https://api.atlassian.com/ex/jira/{cloud_id}/{path}",
            headers={"Authorization": f"Bearer {token}", "Accept": "application/json"},
            params={**params, "startAt": start_at, "maxResults": max_results}

```

```

    )
    data = resp.json()
    items = data.get("issues") or data.get("values") or data.get("users") or []
    results.extend(items)

    total = data.get("total", 0)
    start_at += len(items)
    if start_at >= total or not items:
        break

return results

```

Rate Limit Handling

Provider	Limit	Strategy
GitHub	5000 req/hr per token	Check X-RateLimit-Remaining header; sleep until X-RateLimit-Reset if < 10 remaining
Jira	10 req/s per OAuth token	Add 100ms delay between requests; exponential backoff on 429

```

async def rate_limit_aware_request(client, url, headers, provider):
    while True:
        resp = await client.get(url, headers=headers)

        if provider == "github":
            remaining = int(resp.headers.get("X-RateLimit-Remaining", 1))
            if remaining < 5:
                reset_at = int(resp.headers.get("X-RateLimit-Reset", time.time() + 60))
                await asyncio.sleep(reset_at - time.time() + 1)

        if resp.status_code == 429:
            retry_after = int(resp.headers.get("Retry-After", 60))
            await asyncio.sleep(retry_after)
            continue

    return resp

```

5. Ongoing Ingestion — Webhook Processing

After initial sync, all updates arrive via webhooks. Both GitHub and Jira send POST requests to your endpoint.

Webhook Endpoint Architecture

```

sequenceDiagram
    participant GH as GitHub / Jira
    participant WH as Webhook Endpoint (FastAPI)
    participant SIG as Signature Verifier
    participant IDM as Idempotency Guard (Redis)

```

```

participant S3 as S3 Archive
participant RQ as Redis Streams
participant WK as Processing Worker
participant CG as Cogne

GH->>WH: POST /webhooks/{provider}/{connection_id}
WH->>SIG: Verify HMAC signature
note over SIG: GitHub: X-Hub-Signature-256<br/>Jira: no built-in sig – use secret
alt Signature invalid
    SIG-->>WH: Reject
    WH-->>GH: 401
else Valid
    WH-->>GH: 200 OK (IMMEDIATE – before any processing)
    WH->>IDM: Check key: {provider}:{event_type}:{resource_id}:{delivery_id}
    alt Duplicate
        IDM-->>WH: Skip
    else New event
        IDM->>S3: Archive raw payload
        IDM->>RQ: XADD events {provider, event_type, payload, connection_id}
    end
end

RQ->>WK: XREAD (blocking, group: workers)
WK->>WK: Normalise → resolve entities → extract relations
WK->>CG: add_data_points(entities, dataset=tenant_id)
WK->>RQ: XACK (mark processed)

```

GitHub Webhook Signature Verification

```

import hmac, hashlib

def verify_github_signature(payload_body: bytes, signature_header: str, secret: str)
    """GitHub sends X-Hub-Signature-256: sha256=HASH"""
    if not signature_header or not signature_header.startswith("sha256="):
        return False
    expected = "sha256=" + hmac.new(
        secret.encode(), payload_body, hashlib.sha256
    ).hexdigest()
    return hmac.compare_digest(expected, signature_header)

@app.post("/webhooks/github/{connection_id}")
async def github_webhook(
    connection_id: str,
    request: Request,
    x_hub_signature_256: str = Header(None),
    x_github_event: str = Header(None),
    x_github_delivery: str = Header(None),
):
    payload_body = await request.body()
    conn = db.get(IntegrationConnection, connection_id)
    secret = decrypt(conn.webhook_secret_enc)

    if not verify_github_signature(payload_body, x_hub_signature_256, secret):
        raise HTTPException(401, "Invalid signature")

```

```
# Return 200 immediately – never block the webhook response
background_tasks.add_task(
    enqueue_event,
    provider="github",
    event_type=x_github_event,
    delivery_id=x_github_delivery,
    connection_id=connection_id,
    payload=payload_body
)
return {"status": "accepted"}
```

Jira Webhook Event Handling

Jira does **not** natively sign webhook payloads (unlike GitHub). Two mitigations:

1. Include a **secret token** in the webhook URL path:

```
/webhooks/jira/{connection_id}/{webhook_secret}
```

2. **Whitelist Atlassian IPs** (optional, Atlassian publishes their IP ranges)

```
@app.post("/webhooks/jira/{connection_id}/{secret_token}")
async def jira_webhook(
    connection_id: str,
    secret_token: str,
    request: Request
):
    conn = db.get(IntegrationConnection, connection_id)
    expected_secret = decrypt(conn.webhook_secret_enc)

    if not hmac.compare_digest(secret_token, expected_secret):
        raise HTTPException(401, "Invalid secret token")

    payload = await request.json()
    background_tasks.add_task(
        enqueue_event,
        provider="jira",
        event_type=payload.get("webhookEvent"),
        delivery_id=payload.get("timestamp"), # Jira uses timestamp as unique ID
        connection_id=connection_id,
        payload=payload
    )
    return {"status": "accepted"}
```

6. Event Normalisation — Schema Mapping

Each provider's event payload is mapped to your internal entity schema before writing to Cognition.

GitHub Events → Internal Schema

GitHub Event	GitHub Payload Fields	Internal Entity	Key Mapped Fields
issues.opened / edited / closed	issue.id,title,body,state, user,labels,milestone	WorkItem	title, status, source="github", assignee_id, labels
pull_request.*	pull_request.id,title,state, user,base,head,merged_at	CodeChange	title, status, author_id, repo_id, linked_issue_ids (from body regex)
push	commits[], repository, pusher	Commit	sha, message, author_id, repo_id, timestamp
create (branch/tag)	ref, ref_type, repository	BranchEvent	branch_name, repo_id, actor_id
deployment	deployment.environment, sha, creator	Deployment	env, commit_sha, actor_id, repo_id

```
def normalise_github_issue(payload: dict, connection: IntegrationConnection) -> WorkItem:
    issue = payload["issue"]
    return WorkItem(
        id=f"github:issue:{issue['id']}",
        title=issue["title"],
        status=issue["state"],                # "open" | "closed"
        priority="none",                     # GitHub issues have no native priority
        source="github",
        source_url=issue["html_url"],
        external_id=str(issue["id"]),
        tenant_id=connection.tenant_id,
        assignee_ref=issue.get("assignee", {}).get("login"),
        labels=[l["name"] for l in issue.get("labels", [])],
        updated_at=issue["updated_at"],
        metadata={"index_fields": ["title", "status", "labels"]}
    )
```

Jira Events → Internal Schema

Jira Event	Key Payload Fields	Internal Entity	Key Mapped Fields
jira:issue_created	issue.id, fields.summary, fields.status, fields.assignee, fields.priority	WorkItem	title, status, priority, source="jira", assignee_id
jira:issue_updated	same + changelog.items	WorkItem (update)	Only changed fields — use changelog to detect field diffs
comment_created	comment.body, comment.author, issue.id	Comment	content, author_id, work_item_id
sprint_started	sprint.id, name, startDate, endDate	Sprint	name, status, start_date, end_date

Jira Event	Key Payload Fields	Internal Entity	Key Mapped Fields
sprint_closed	sprint.id,completedDate	Sprint (update)	status="closed", completed_at

```
def normalise_jira_issue(payload: dict, connection: IntegrationConnection) -> WorkItem:
    issue = payload["issue"]
    fields = issue["fields"]
    assignee = fields.get("assignee") or {}
    priority = (fields.get("priority") or {}).get("name", "Medium")

    return WorkItem(
        id=f"jira:issue:{issue['id']}",
        title=fields["summary"],
        status=fields["status"]["name"],
        priority=priority,
        source="jira",
        source_url=f"{connection.site_url}/browse/{issue['key']}",
        external_id=issue["id"],
        external_key=issue["key"],          # Jira-specific: "PROJ-123"
        tenant_id=connection.tenant_id,
        assignee_ref=assignee.get("emailAddress"),
        sprint_ref=_extract_sprint_id(fields),
        updated_at=fields["updated"],
        metadata={"index_fields": ["title", "status", "priority", "external_key"]}
    )
```

7. Polling Safety Net

Webhooks can miss events (network errors, provider outages, Jira webhook expiry). A polling scheduler runs concurrently to catch any gaps.

```
flowchart TD
    SCHED[Polling Scheduler\nRuns every N minutes per connection]

    subgraph GITHUB_POLL["GitHub Polling (every 30 min)"]
        GP1[GET /repos/:owner/:repo/issues\n?since=last_poll_cursor&state=all]
        GP2[GET /repos/:owner/:repo/pulls\n?since=last_poll_cursor]
        GP3[Update cursor to current time]
    end

    subgraph JIRA_POLL["Jira Polling (every 10 min)"]
        JP1[GET /rest/api/3/search\n?jql=updated >= last_poll_cursor\n&fields=all&maxResults=10]
        JP2[GET /rest/agile/1.0/board/:id/sprint\n?state=active,future]
        JP3[Update cursor to current time]
    end

    SCHED --> GITHUB_POLL
    SCHED --> JIRA_POLL
    GITHUB_POLL --> EMIT[Emit events to Redis Streams\n(same pipeline as webhooks)]
    JIRA_POLL --> EMIT
```

```

# Jira incremental poll – cursor-based using updated_at
async def poll_jira_updates(connection: IntegrationConnection):
    cursor = connection.last_poll_cursor or (datetime.utcnow() - timedelta(hours=24))
    cursor_jira_fmt = cursor.replace("T", " ").split(".")[0] # Jira JQL format: "20

    issues = await paginate_jira(
        "rest/api/3/search",
        token=decrypt(connection.access_token_enc),
        cloud_id=connection.cloud_id,
        params={"jql": f'updated >= "{cursor_jira_fmt}"',
                "fields": "summary,status,assignee,priority,sprint,updated"}
    )

    for issue in issues:
        await enqueue_event(
            provider="jira",
            event_type="jira:issue_updated",
            delivery_id=f"poll:{issue['id']}:{issue['fields']['updated']}",
            connection_id=connection.id,
            payload={"issue": issue}
        )

    connection.last_poll_cursor = datetime.utcnow().isoformat()
    db.commit()

```

8. Ingestion into Cognee

After normalisation, entities are written to Cognee using `add_data_points()`, scoped to the tenant's dataset. This gives you full control – no LLM auto-extraction for structured provider data.

```

from cognee.api.v1.add.add_data_points import add_data_points
from cognee.modules.pipelines.operations.run_tasks import PipelineContext

async def ingest_to_cognee(entities: list, tenant_id: str):
    """Write normalised entities + relationships to Cognee knowledge graph."""

    # Scope all operations to tenant's isolated dataset
    context = PipelineContext(dataset_name=f"tenant_{tenant_id}")

    # Entities already have relationships embedded as SkipValidation fields
    await add_data_points(entities, pipeline_context=context)

    # Example: after ingesting a CodeChange that has a linked WorkItem
    # Cognee automatically creates the edge:
    # (CodeChange) --resolves--> (WorkItem)
    # because WorkItem is declared as a typed field on CodeChange

    async def process_github_pr_event(payload: dict, connection: IntegrationConnection):
        pr = normalise_github_pr(payload, connection)
        author = await resolve_or_create_person(
            github_login=payload["pull_request"]["user"]["login"],

```

```

        tenant_id=connection.tenant_id
    )
    pr.author = author # Sets the edge: PR --authored_by--> Person

    # Link to Jira issues mentioned in PR body/title (e.g., "PROJ-123")
    jira_refs = extract_jira_refs(payload["pull_request"]["body"] or "")
    for ref in jira_refs:
        work_item = db.get_work_item_by_key(ref, connection.tenant_id)
        if work_item:
            pr.linked_items.append(work_item) # Edge: PR --resolves--> WorkItem

    await ingest_to_cognee([pr, author] + pr.linked_items, connection.tenant_id)

```

9. Complete Ingestion Flow Summary

```

flowchart TD
    subgraph AUTH["Step 1: Authorization"]
        U[User clicks Connect GitHub/Jira]
        OA[OAuth 2.0 PKCE flow]
        TK[Tokens stored encrypted in Postgres]
        U --> OA --> TK
    end

    subgraph SETUP["Step 2: Connection Setup"]
        WR[Register webhooks on provider]
        AS[Mark connection = active]
        TK --> WR --> AS
    end

    subgraph SNAP["Step 3: Initial Snapshot"]
        PG[Paginate all historical data\nrepos, issues, PRs, sprints]
        S3W[Archive raw JSON to S3]
        NR[Normalise + entity resolve]
        CW[Write to Cognee via add_data_points]
        AS --> PG --> S3W --> NR --> CW
    end

    subgraph ONGOING["Step 4: Ongoing Ingestion"]
        direction LR
        WH2[Webhook events\nreal-time < 5s]
        PL[Polling\nevery 10-30 min]
        EQ2[Redis Streams\nEvent Queue]
        WK2[Processing Workers\nnormalise + ingest]
        CG2[Cognee Knowledge Graph]
        WH2 --> EQ2
        PL --> EQ2
        EQ2 --> WK2 --> CG2
    end

    SNAP --> ONGOING

```


10. Edge Cases and Gotchas

Issue	What happens	How to handle
Jira webhook expires (30 days)	Events silently stop arriving	Scheduled job refreshes webhooks every 25 days via <code>PUT /rest/api/3/webhook/refresh</code>
Jira token expires (1 hour)	API calls return 401	Refresh middleware auto-renews using <code>offline_access</code> refresh token (rotating, 90-day inactivity expiry)
GitHub rate limit (5000 req/hr)	403 responses during initial sync	Check <code>X-RateLimit-Remaining</code> ; pause until <code>X-RateLimit-Reset</code>
Duplicate events	Both webhook + poll fire for same update	Redis idempotency key with 48h TTL deduplicates before processing
Missed webhooks (provider outage)	Stale knowledge graph	Polling safety net catches within polling interval (10-30 min)
User revokes access	Token becomes invalid permanently	403 <code>invalid_grant</code> on refresh → mark connection <code>revoked</code> , notify user to reconnect via UI
PR references Jira issue	Cross-source edge needed	Regex <code>[A-Z]+-\d+</code> on PR title/body; look up <code>WorkItem</code> by <code>external_key</code> to create <code>resolves</code> edge
Same person in both systems	Duplicate Person nodes	Resolve via <code>email</code> field: GitHub user email == Jira user <code>emailAddress</code> . Merge into single canonical <code>Person</code> node
Org has thousands of repos	Initial sync takes too long	Chunk into batches of 10 repos; process concurrently with semaphore limiting to 3 concurrent; update progress in Postgres

Document generated: June 28, 2026 — Bondhu Tech Enterprise AI Platform